EmployeePortal – Clean, scalable .NET 9 blueprint

This blueprint gives you a professional, API-first architecture you can grow into web + mobile clients. It uses **Clean Architecture** layering with a crisp separation of concerns and production-grade defaults (auth, validation, logging, versioning, OpenAPI, EF Core, testing, CI stubs).

> **Key ideas** - Domain in **Core** (pure C# – entities, value objects, enums, contracts) - Use cases in **Services** (CQRS/MediatR optional) – zero infrastructure details - Persistence and adapters in **Infrastructure** (EF Core, external services) - **Api** exposes versioned REST; **WebApp** is a thin UI consuming the Api - Strict dependencies: Core → Services → Infrastructure → Api; WebApp → Api (HTTP)

---

1) Solution layout

```
EmployeePortal
├─ src
│  ├─ EmployeePortal.Core                # Domain: Entities, ValueObjects,
Enums, Abstractions
│  ├─ EmployeePortal.Services            # Application: DTOs, Commands/Queries,
Validators, Mapping
│  ├─ EmployeePortal.Infrastructure      # EF Core, Repositories, Identity,
Adapters
│  ├─ EmployeePortal.Api                 # ASP.NET Core 9 Web API (versioned)
│  └─ EmployeePortal.WebApp              # (Optional) MVC/Razor UI that calls
the Api
│
├─ tests
│  ├─ EmployeePortal.UnitTests           # Core + Services unit tests
│  ├─ EmployeePortal.IntegrationTests    # EF Core + Api integration (SQLite/
Testcontainers)
│  └─ EmployeePortal.FunctionalTests     # End-to-end happy paths
│
└─ build                                 # Dockerfile, GH Actions/Az DevOps
yaml, scripts
```

Create via CLI

```
mkdir EmployeePortal && cd EmployeePortal
mkdir src tests build

# Projects
dotnet new classlib -n EmployeePortal.Core -o src/EmployeePortal.Core -f net9.0
```

```
dotnet new classlib -n EmployeePortal.Services -o src/EmployeePortal.Services -
f net9.0
dotnet new classlib -n EmployeePortal.Infrastructure -o src/
EmployeePortal.Infrastructure -f net9.0
dotnet new webapi   -n EmployeePortal.Api -o src/EmployeePortal.Api -f net9.0
dotnet new mvc      -n EmployeePortal.WebApp -o src/EmployeePortal.WebApp -f
net9.0

# Tests
dotnet new xunit -n EmployeePortal.UnitTests -o tests/EmployeePortal.UnitTests -
f net9.0
dotnet new xunit -n EmployeePortal.IntegrationTests -o tests/
EmployeePortal.IntegrationTests -f net9.0

# Solution + references
dotnet new sln -n EmployeePortal

dotnet sln add src/EmployeePortal.Core src/EmployeePortal.Services src/
EmployeePortal.Infrastructure src/EmployeePortal.Api src/EmployeePortal.WebApp
tests/EmployeePortal.UnitTests tests/EmployeePortal.IntegrationTests

dotnet add src/EmployeePortal.Services reference src/EmployeePortal.Core
dotnet add src/EmployeePortal.Infrastructure reference src/
EmployeePortal.Services src/EmployeePortal.Core
dotnet add src/EmployeePortal.Api reference src/EmployeePortal.Services src/
EmployeePortal.Infrastructure
```

Common engineering settings

Create `Directory.Build.props` at repo root:

```
<Project>
  <PropertyGroup>
    <LangVersion>preview</LangVersion>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

---

2) Core (Domain) – models, interfaces, enums

Project: `EmployeePortal.Core`

Folders: `Entities/`, `ValueObjects/`, `Enums/`, `Abstractions/`

```
// Enums/EmploymentStatus.cs
namespace EmployeePortal.Core.Enums;
public enum EmploymentStatus { Active = 1, OnLeave = 2, Terminated = 3 }
```

```
// ValueObjects/Email.cs
namespace EmployeePortal.Core.ValueObjects;
public sealed class Email
{
    public string Value { get; }
    public Email(string value)
    {
        if (string.IsNullOrWhiteSpace(value) || !value.Contains('@'))
            throw new ArgumentException("Invalid email.");
        Value = value.Trim();
    }
    public override string ToString() => Value;
}
```

```
// Entities/Employee.cs
using EmployeePortal.Core.Enums;
using EmployeePortal.Core.ValueObjects;

namespace EmployeePortal.Core.Entities;

public sealed class Employee
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public string FirstName { get; private set; }
    public string LastName  { get; private set; }
    public Email Email      { get; private set; }
    public EmploymentStatus Status { get; private set; } =
EmploymentStatus.Active;
    public DateOnly HireDate { get; private set; }

    private Employee() {}
    public Employee(string firstName, string lastName, Email email, DateOnly
hireDate)
    {
        FirstName = string.IsNullOrWhiteSpace(firstName) ? throw new
ArgumentException("FirstName required") : firstName.Trim();
        LastName  = string.IsNullOrWhiteSpace(lastName)  ? throw new
ArgumentException("LastName required")  : lastName.Trim();
```

```
        Email     = email ?? throw new ArgumentNullException(nameof(email));
        HireDate  = hireDate;
    }

    public void Terminate(DateOnly on) => Status = EmploymentStatus.Terminated;
}
```

```
// Abstractions/Repositories/IEmployeeRepository.cs
using EmployeePortal.Core.Entities;

namespace EmployeePortal.Core.Abstractions.Repositories;

public interface IEmployeeRepository
{
    Task<Employee?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<bool> EmailExistsAsync(string email, CancellationToken ct = default);
    Task AddAsync(Employee employee, CancellationToken ct = default);
    Task UpdateAsync(Employee employee, CancellationToken ct = default);
    Task<(IReadOnlyList<Employee> Items, int Total)> SearchAsync(string? q, int
page, int pageSize, CancellationToken ct = default);
}
```

```
// Abstractions/IUnitOfWork.cs
namespace EmployeePortal.Core.Abstractions;
public interface IUnitOfWork { Task<int> SaveChangesAsync(CancellationToken ct
= default); }
```

---

3) Services (Application) – DTOs, use-cases, validation, mapping

Project: `EmployeePortal.Services`

NuGets

```
dotnet add src/EmployeePortal.Services package MediatR
dotnet add src/EmployeePortal.Services package FluentValidation
dotnet add src/EmployeePortal.Services package AutoMapper
```

Folders: `DTOs/`, `Employees/Commands/`, `Employees/Queries/`, `Validation/`, `Mapping/`

```
// DTOs/EmployeeDto.cs
namespace EmployeePortal.Services.DTOs;
public sealed record EmployeeDto(Guid Id, string FirstName, string LastName,
string Email, string Status, DateOnly HireDate);
```

```
// Employees/Commands/CreateEmployee.cs
using MediatR;
using EmployeePortal.Services.DTOs;
public sealed record CreateEmployee(string FirstName, string LastName, string
Email, DateOnly HireDate) : IRequest<EmployeeDto>;
```

```
// Employees/Commands/CreateEmployeeHandler.cs
using AutoMapper;
using MediatR;
using EmployeePortal.Core.Abstractions;
using EmployeePortal.Core.Abstractions.Repositories;
using EmployeePortal.Core.Entities;
using EmployeePortal.Core.ValueObjects;
using EmployeePortal.Services.DTOs;

public sealed class CreateEmployeeHandler(IEmployeeRepository repo, IUnitOfWork
uow, IMapper mapper) : IRequestHandler<CreateEmployee, EmployeeDto>
{
    public async Task<EmployeeDto> Handle(CreateEmployee req, CancellationToken
ct)
    {
        if (await repo.EmailExistsAsync(req.Email, ct))
            throw new InvalidOperationException("Email already exists");
        var emp = new Employee(req.FirstName, req.LastName, new
Email(req.Email), req.HireDate);
        await repo.AddAsync(emp, ct);
        await uow.SaveChangesAsync(ct);
        return mapper.Map<EmployeeDto>(emp);
    }
}
```

```
// Employees/Queries/GetEmployeeById.cs
using MediatR;
using EmployeePortal.Services.DTOs;
public sealed record GetEmployeeById(Guid Id) : IRequest<EmployeeDto?>;
```

```
// Employees/Queries/GetEmployeeByIdHandler.cs
using AutoMapper;
using MediatR;
using EmployeePortal.Core.Abstractions.Repositories;
using EmployeePortal.Services.DTOs;

public sealed class GetEmployeeByIdHandler(IEmployeeRepository repo, IMapper
mapper) : IRequestHandler<GetEmployeeById, EmployeeDto?>
{
    public async Task<EmployeeDto?> Handle(GetEmployeeById req,
CancellationToken ct)
    {
        var entity = await repo.GetByIdAsync(req.Id, ct);
        return entity is null ? null : mapper.Map<EmployeeDto>(entity);
    }
}
```

```
// Employees/Queries/SearchEmployees.cs
using MediatR;
using EmployeePortal.Services.DTOs;
public sealed record SearchEmployees(string? Q, int Page = 1, int PageSize =
20) : IRequest<PagedResult<EmployeeDto>>;
public sealed record PagedResult<T>(IReadOnlyList<T> Items, int Total, int
Page, int PageSize);
```

```
// Employees/Queries/SearchEmployeesHandler.cs
using AutoMapper;
using MediatR;
using EmployeePortal.Core.Abstractions.Repositories;
using EmployeePortal.Services.DTOs;

public sealed class SearchEmployeesHandler(IEmployeeRepository repo, IMapper
mapper) : IRequestHandler<SearchEmployees, PagedResult<EmployeeDto>>
{
    public async Task<PagedResult<EmployeeDto>> Handle(SearchEmployees req,
CancellationToken ct)
    {
        var (items, total) = await repo.SearchAsync(req.Q, req.Page,
req.PageSize, ct);
        var dtos = items.Select(mapper.Map<EmployeeDto>).ToList();
        return new(dtos, total, req.Page, req.PageSize);
    }
}
```

```
// Validation/CreateEmployeeValidator.cs
using FluentValidation;
public sealed class CreateEmployeeValidator : AbstractValidator<CreateEmployee>
{
    public CreateEmployeeValidator()
    {
        RuleFor(x => x.FirstName).NotEmpty().MaximumLength(100);
        RuleFor(x => x.LastName).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Email).NotEmpty().EmailAddress().MaximumLength(256);
        RuleFor(x =>
x.HireDate).LessThanOrEqualTo(DateOnly.FromDateTime(DateTime.UtcNow.Date));
    }
}
```

```
// Mapping/MappingProfile.cs
using AutoMapper;
using EmployeePortal.Core.Entities;
using EmployeePortal.Services.DTOs;

public sealed class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Employee, EmployeeDto>()
            .ForMember(d => d.Email, m => m.MapFrom(s => s.Email.Value))
            .ForMember(d => d.Status, m => m.MapFrom(s => s.Status.ToString()));
    }
}
```

```
// DependencyInjection.cs (Services)
using Microsoft.Extensions.DependencyInjection;
using MediatR;
using AutoMapper;
using System.Reflection;

namespace EmployeePortal.Services;

public static class DependencyInjection
{
    public static IServiceCollection AddApplicationServices(this
IServiceCollection services)
    {
        services.AddMediatR(cfg =>
cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly()));
        services.AddAutoMapper(Assembly.GetExecutingAssembly());
```

```
        return services;
    }
}
```

---

4) Infrastructure – EF Core, repositories, unit of work

Project: `EmployeePortal.Infrastructure`

NuGets

```
dotnet add src/EmployeePortal.Infrastructure package
Microsoft.EntityFrameworkCore
dotnet add src/EmployeePortal.Infrastructure package
Microsoft.EntityFrameworkCore.SqlServer
dotnet add src/EmployeePortal.Infrastructure package
Microsoft.EntityFrameworkCore.Design
```

Code

```
// Persistence/AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using EmployeePortal.Core.Entities;

namespace EmployeePortal.Infrastructure.Persistence;

public sealed class AppDbContext(DbContextOptions<AppDbContext> options) :
DbContext(options)
{
    public DbSet<Employee> Employees => Set<Employee>();
    protected override void OnModelCreating(ModelBuilder b)
    {
        b.Entity<Employee>(e =>
        {
            e.HasKey(x => x.Id);
            e.Property(x => x.FirstName).HasMaxLength(100).IsRequired();
            e.Property(x => x.LastName).HasMaxLength(100).IsRequired();
            e.OwnsOne(x => x.Email, nb => nb.Property(p =>
p.Value).HasColumnName("Email").HasMaxLength(256).IsRequired());
        });
    }
}
```

```
// Repositories/EmployeeRepository.cs
using Microsoft.EntityFrameworkCore;
using EmployeePortal.Core.Abstractions.Repositories;
using EmployeePortal.Core.Entities;
using EmployeePortal.Infrastructure.Persistence;

namespace EmployeePortal.Infrastructure.Repositories;

public sealed class EmployeeRepository(AppDbContext db) : IEmployeeRepository
{
    public Task<Employee?> GetByIdAsync(Guid id, CancellationToken ct = default)
        => db.Employees.AsNoTracking().FirstOrDefaultAsync(x => x.Id == id, ct);

    public Task<bool> EmailExistsAsync(string email, CancellationToken ct =
default)
        => db.Employees.AsNoTracking().AnyAsync(x => x.Email.Value == email,
ct);

    public Task AddAsync(Employee employee, CancellationToken ct = default)
        => db.Employees.AddAsync(employee, ct).AsTask();

    public Task UpdateAsync(Employee employee, CancellationToken ct = default)
    { db.Employees.Update(employee); return Task.CompletedTask; }

    public async Task<(IReadOnlyList<Employee> Items, int Total)>
SearchAsync(string? q, int page, int pageSize, CancellationToken ct = default)
    {
        var query = db.Employees.AsNoTracking();
        if (!string.IsNullOrWhiteSpace(q))
            query = query.Where(x => x.FirstName.Contains(q) ||
x.LastName.Contains(q) || x.Email.Value.Contains(q));
        var total = await query.CountAsync(ct);
        var items = await query.OrderBy(x => x.LastName).ThenBy(x =>
x.FirstName)
            .Skip((page - 1) * pageSize).Take(pageSize).ToListAsync(ct);
        return (items, total);
    }
}
```

```
// Persistence/UnitOfWork.cs
using EmployeePortal.Core.Abstractions;
using EmployeePortal.Infrastructure.Persistence;

namespace EmployeePortal.Infrastructure.Persistence;

public sealed class UnitOfWork(AppDbContext db) : IUnitOfWork
```

```
{ public Task<int> SaveChangesAsync(CancellationToken ct = default) =>
db.SaveChangesAsync(ct); }
```

```
// DependencyInjection.cs (Infrastructure)
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using EmployeePortal.Core.Abstractions;
using EmployeePortal.Core.Abstractions.Repositories;
using EmployeePortal.Infrastructure.Persistence;
using EmployeePortal.Infrastructure.Repositories;

namespace EmployeePortal.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(this IServiceCollection
services, string connectionString)
    {
        services.AddDbContext<AppDbContext>(opt =>
opt.UseSqlServer(connectionString));
        services.AddScoped<IEmployeeRepository, EmployeeRepository>();
        services.AddScoped<IUnitOfWork, UnitOfWork>();
        return services;
    }
}
```

---

5) Api – versioned, documented, validated

Project: `EmployeePortal.Api`

NuGets

```
dotnet add src/EmployeePortal.Api package Swashbuckle.AspNetCore
dotnet add src/EmployeePortal.Api package FluentValidation.AspNetCore
```

Program.cs

```
using EmployeePortal.Services;
using EmployeePortal.Infrastructure;
using FluentValidation;
using Microsoft.AspNetCore.Mvc;
```

```
var builder = WebApplication.CreateBuilder(args);

var conn = builder.Configuration.GetConnectionString("Default")!;

builder.Services.AddApplicationServices();
builder.Services.AddInfrastructure(conn);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddApiVersioning(o =>
{
    o.AssumeDefaultVersionWhenUnspecified = true;
    o.DefaultApiVersion = new ApiVersion(1, 0);
    o.ReportApiVersions = true;
});

builder.Services.AddFluentValidationAutoValidation();
builder.Services.AddValidatorsFromAssemblyContaining<CreateEmployeeValidator>();

builder.Services.AddCors(o => o.AddPolicy("AllowAll", p =>
p.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod()));

var app = builder.Build();

app.UseCors("AllowAll");
app.UseHttpsRedirection();
app.UseSwagger();
app.UseSwaggerUI();
app.MapControllers();
app.Run();
```

Controller

```
// Controllers/V1/EmployeesController.cs
using MediatR;
using Microsoft.AspNetCore.Mvc;
using EmployeePortal.Services;

namespace EmployeePortal.Api.Controllers.V1;

[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/employees")]
```

```
public sealed class EmployeesController(IMediator mediator) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreateEmployee cmd,
CancellationToken ct)
    {
        var dto = await mediator.Send(cmd, ct);
        return CreatedAtAction(nameof(GetById), new { id = dto.Id, version =
HttpContext.GetRequestedApiVersion()?.ToString() }, dto);
    }

    [HttpGet("{id:guid}")]
    public async Task<IActionResult> GetById(Guid id, CancellationToken ct)
    {
        var dto = await mediator.Send(new GetEmployeeById(id), ct);
        return dto is null ? NotFound() : Ok(dto);
    }

    [HttpGet]
    public async Task<IActionResult> Search([FromQuery] string? q, [FromQuery]
int page = 1, [FromQuery] int pageSize = 20, CancellationToken ct = default)
        => Ok(await mediator.Send(new SearchEmployees(q, page, pageSize), ct));
}
```

appsettings.Development.json

```
{
  "ConnectionStrings": {
    "Default":
"Server=localhost;Database=EmployeePortal;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

---

6) WebApp – thin UI consuming the Api

Project: `EmployeePortal.WebApp` (MVC or Razor Pages). Keep it simple: call the Api via HttpClient.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddHttpClient("Api", c => c.BaseAddress = new("https://
localhost:5001/"));
var app = builder.Build();
```

```
app.UseHttpsRedirection();
app.UseStaticFiles();
app.MapDefaultControllerRoute();
app.Run();
```

```
// Controllers/EmployeesController.cs (WebApp)
using Microsoft.AspNetCore.Mvc;
using System.Net.Http.Json;

public sealed class EmployeesController(IHttpClientFactory http) : Controller
{
    public async Task<IActionResult> Index(string? q, int page = 1)
    {
        var client = http.CreateClient("Api");
        var data = await client.GetFromJsonAsync<object>($"api/v1/employees?
q={q}&page={page}");
        return View(data);
    }
}
```

---

7) Migrations & database

From the `src/EmployeePortal.Api` folder:

```
# Ensure Infrastructure has design-time factory if needed, or run from Api with
reference
 dotnet tool install --global dotnet-ef
 dotnet ef migrations add InitialCreate -p ../EmployeePortal.Infrastructure -
s . -o Persistence/Migrations
 dotnet ef database update -p ../EmployeePortal.Infrastructure -s .
```

Optional design-time factory in Infrastructure:

```
// Persistence/DesignTimeFactory.cs
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;

namespace EmployeePortal.Infrastructure.Persistence;
public sealed class DesignTimeFactory :
IDesignTimeDbContextFactory<AppDbContext>
{
    public AppDbContext CreateDbContext(string[] args)
```

```
    {
        var opts = new DbContextOptionsBuilder<AppDbContext>()
            .UseSqlServer("Server=.;Database=EmployeePortal;Trusted_Connection=True;TrustServerCe
            .Options;
        return new AppDbContext(opts);
    }
}
```

---

8) Cross-cutting: error handling, logging, auth (stubs)

- **Error handling**: add a global exception middleware in Api to turn exceptions into RFC7807 ProblemDetails.
- **Logging**: configure Serilog (sink to console / Seq) at Api startup.
- **Auth**: add JWT bearer auth when needed; decorate controllers with `[Authorize]`.

Example ProblemDetails middleware:

```
// Api/Middleware/ProblemDetailsMiddleware.cs
public class ProblemDetailsMiddleware(RequestDelegate next,
ILogger<ProblemDetailsMiddleware> logger)
{
    public async Task Invoke(HttpContext ctx)
    {
        try { await next(ctx); }
        catch (Exception ex)
        {
            logger.LogError(ex, "Unhandled error");
            ctx.Response.StatusCode = StatusCodes.Status500InternalServerError;
            await ctx.Response.WriteAsJsonAsync(new { title = "Server error",
detail = ex.Message });
        }
    }
}
```

Register in `Program.cs` before `MapControllers()`:

```
app.UseMiddleware<ProblemDetailsMiddleware>();
```

---

9) Testing layout

```
// Unit tests (Core/Services)
EmployeePortal.UnitTests/
  Employees/
    CreateEmployeeHandlerTests.cs

// Integration tests (EF + Api)
EmployeePortal.IntegrationTests/
  EmployeesApiTests.cs (use WebApplicationFactory)
```

---

10) Next steps & scaling

- Add aggregates (Departments, Roles, Leave, Payroll) as separate folders with CQRS handlers.
- Introduce **Outbox** pattern for integration events.
- Add **OpenAPI client** generation for WebApp/Mobile (NSwag).
- Add **Redis** caching for queries; add **Hangfire/Quartz** for background jobs.
- Split read models if needed (CQRS) and project to search index (e.g., Elastic/Azure AI Search).
- Containerize with Docker, add CI pipeline (GH Actions/Azure DevOps) and IaC.

---

11) Quick start

```
# 1) Restore & build
dotnet restore && dotnet build -v minimal
# 2) Run API (Swagger at /swagger)
dotnet run --project src/EmployeePortal.Api
# 3) Run WebApp (consumes API)
dotnet run --project src/EmployeePortal.WebApp
```

This gives you a clean, extensible base with **Core/Services/WebApp** naming you asked for, plus `Infrastructure` + `Api` to keep concerns isolated. Expand features by adding new Command/Query + Handler + Repo methods without leaking infrastructure into the domain.